Mobile Application Development with Flutter

Lecture 51. Firebase with Flutter

Introduction

- **Firebase** is Google's Backend-as-a-Service (BaaS) platform.
- It lets you build apps without manually creating your own backend server, APIs and databases from scratch
- For Flutter, Firebase is connected through **FlutterFire plugins**

🧠 Core Concepts of Firebase

1. Serverless Architecture
2. Realtime Data
3. NoSQL Database
4. SDK-Based Communication

1. Serverless Architecture

- We don’t manage servers — the Firebase handles everything for us
- 🔁 Traditional Architecture (with servers)
  - Normally, apps work like this:
  - App → Backend Server → Database
- Developer must:
  - Create server (Node.js, PHP, etc.)
  - Deploy it
  - Handle scaling (more users → more servers)
  - Maintain uptime
  - Fix crashes

1. Serverless Architecture

- ⚡ Serverless Architecture (Firebase way)
  - App → Firebase Services (No server management)
- Developer just:
  - Write app code
  - Call Firebase service
- 👉 Firebase handles:
  - Servers
  - Scaling
  - Security
  - Maintenance

⚙️ **Real Example**

```
FirebaseFirestore.instance.collection('users').add({
  'name': 'Ali'
});
```

2. Realtime Data

- Realtime data means:
  - When data changes in the database, the app automatically updates instantly — without refreshing
- Simple definition:
  - Realtime = Data syncs instantly across all users/devices

🔁 Traditional apps (NO realtime)

- App → Fetch Data → Show Data

⚡ Firebase Realtime behavior

- Database changes → App updates automatically

3. NoSQL Database

- Firebase databases are NoSQL
- In SQL data looks like

```
Users Table
----------------
id | name | age
```

- But Firebase doesn't use tables. Instead, it uses:
  - Collection → Document → Fields

🔁 What replaces tables in Firebase?

👉 Collections act like tables

👉 Documents act like rows

👉 Fields act like columns

```
users (collection)
  └── user1 (document)
       ├── name: "Ali"
       └── age: 22
```

3. NoSQL Database

- Important Concept Difference
- SQL:
  - Fixed structure (schema)
  - Must define columns first
- Firebase:
  - Flexible structure
  - Each document can have different fields
- Example:
  - user1 → {name: "Ali", age: 22}
  - user2 → {name: "Sara", city: "Lahore"}

3. NoSQL Database

- How can we add tables in Firebase?
  - We **don't create tables**
  - We create **collections**
- Example in Flutter:

```
FirebaseFirestore.instance.collection('users');
```

- This is equivalent to:
  - Creating a "table" called users

4. SDK-Based Communication

- SDK = Software Development Kit
- So when we say:
  - Firebase uses SDK-based communication
- It means:
  - 👉 Our app talks to Firebase using **pre-built libraries (SDKs)**
  - 👉 We do NOT manually create APIs or HTTP requests

4. SDK-Based Communication

🔁 Traditional way (without Firebase)
  - Flutter App → API (Node/PHP) → Database
- We must:
  - Build REST APIs
  - Handle requests/responses
  - Manage backend server
- Example (manual API call):

```
http.get("https://myapi.com/users");
```

4. SDK-Based Communication

- Firebase way (SDK-based)
  - Flutter App → Firebase SDK → Firebase Services
- Example:

```
FirebaseFirestore.instance.collection('users').get();
```

- 👉 No API
- 👉 No backend code
- 👉 No server handling

📦 Firebase Services

1. Firebase Authentication
2. Cloud Firestore
3. Firebase Storage
4. Firebase Cloud Functions
5. Firebase Cloud Messaging (FCM)
6. Firebase Hosting
7. Firebase Analytics
8. Firebase Crashlytics

1. Firebase Authentication

- Purpose:
  - Handles user login/signup securely
- Features:
  - Email & password login
  - Google login
  - Facebook login
  - Phone OTP login
- Why important:
  - No need to build login system manually
  - Secure and production-ready

2. Cloud Firestore

- Purpose:
  - Main database for storing app data
- Structure:
  - Collection → Document → Fields
- Key Features:
  - Realtime updates
  - Offline support
  - Scalable
  - Flexible schema (no fixed structure)

```
products (collection)
  └── product1 (document)
        ├── name: "Laptop"
        ├── price: 1000
        └── inStock: true
```

3. Firebase Storage

- Purpose:
  - Store large files like images, videos, PDFs
- Example:
  - Uploading profile image
  - storage → users → user123 → profile.jpg
- Why needed:
  - Firestore is NOT for large files — only for data

4. Firebase Cloud Functions

- Purpose:
  - Run backend code automatically
- Example:
  - Send email after signup
  - Process payments
  - Validate data
- Key idea:
  - You write backend logic without managing servers

5. Firebase Cloud Messaging (FCM)

- Purpose
  - Send push notifications
- Example:
  - New message received
  - Order shipped
- Types:
  - Foreground notifications
  - Background notifications

6. Firebase Hosting

- Purpose:
  - Deploy websites/web apps
- Example:
  - Flutter web app hosting

7. Firebase Analytics

- Purpose:
  - Track user behavior
- Example:
  - Screen views
  - Button clicks
  - App usage time

8. Firebase Crashlytics

- Purpose:
  - Track app crashes
- Example:
  - Shows which screen crashed
  - Error logs

How Firebase Services Work Together

- User signs up → Firebase Auth
- User data saved → Firestore
- Profile image → Storage
- Notification → FCM
- Error tracking → Crashlytics